

Processamento de Linguagem Natural e Séries Temporais

Eduardo Adame

Ciência de Dados

29 de maio de 2026



Introdução ao PLN

O que é PLN?

- Campo que estuda como computadores podem **compreender, gerar e manipular** linguagem humana
- Intersecção de linguística, estatística e aprendizado de máquina
- A linguagem é **ambígua, contextual e evolutiva** — o que torna o problema difícil

Por que é difícil?

- “Eu vi o homem com o telescópio” — quem tem o telescópio?
- Ironia, sarcasmo, metáforas
- Conhecimento de mundo implícito
- Línguas morfologicamente ricas (português, árabe, finlandês)

Tarefas clássicas de PLN

- **Tokenização e segmentação** de sentenças
- **POS Tagging** — classe gramatical de cada palavra
- **Parsing** — estrutura sintática da frase
- **NER** — reconhecimento de entidades nomeadas
- **Análise de sentimento**
- **Tradução automática**
- **Sumarização** e geração de texto
- **Resposta a perguntas (QA)**





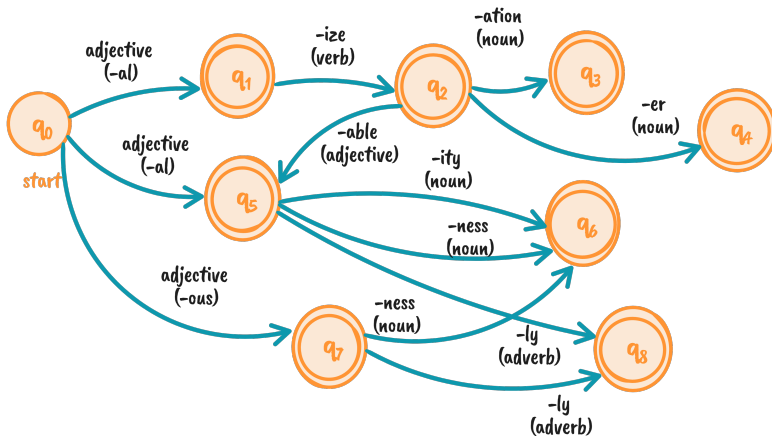
Morfologia computacional

- Estuda a **estrutura interna das palavras**: morfemas, raízes, afixos
- “infelizmente” = in- + feliz + -mente
- Fundamental para lematização e análise morfológica

Autômatos de estados finitos (FSA)

- Modelo matemático que **reconhece padrões** em sequências de símbolos
- FSA determinístico: cada estado tem no máximo uma transição por símbolo
- Transdutor (FST): mapeia uma sequência em outra — ideal para **stemming e lematização**
- Ex.: corr + er → lema correr;
corri → correr

Morfologia e estados finitos





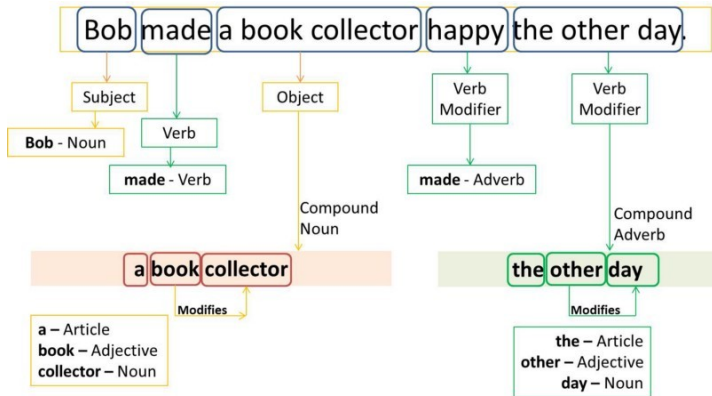
O que é?

- Atribuir a cada token sua **classe gramatical**: substantivo, verbo, adjetivo, ...
- Base para parsing, NER e análise de dependências
- Problema ambíguo: “banco” pode ser substantivo ou verbo

Abordagens

- **Baseada em regras**: dicionário + regras contextuais (Brill Tagger)
- **Estatística**: HMM, CRF — modela sequências de tags
- **Redes neurais**: BiLSTM-CRF, BERT fine-tuned — estado da arte

POS Tagging — etiquetagem morfosintática



Listing 1: Tokenização, POS e NER com spaCy

```
1 import spacy # pip install spacy + download pt_core_news_sm
2
3 nlp = spacy.load("pt_core_news_sm")
4 doc = nlp("O presidente do Brasil visitou Lisboa em maio de 2026.")
5
6 # POS Tagging
7 for token in doc:
8     print(f"{token.text:15s} {token.pos_:8s} {token.lemma_}")
9
10 # Named Entity Recognition (NER)
11 for ent in doc.ents:
12     print(f"{ent.text:20s} → {ent.label_}")
13
14 # Dependências sintáticas
15 for token in doc:
16     print(f"{token.text} --[{token.dep_}]--> {token.head.text}")
```



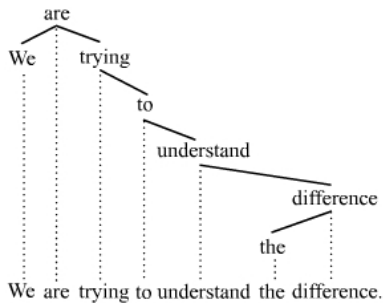
Gramáticas livres de contexto (CFG)

- Regras da forma $A \rightarrow \alpha$: um não-terminal gera uma sequência
- $S \rightarrow NP VP$; $NP \rightarrow DET N$; ...
- Produz **árvore de constituintes** (sintagmas)
- Ambiguidade estrutural: múltiplas árvores para a mesma frase

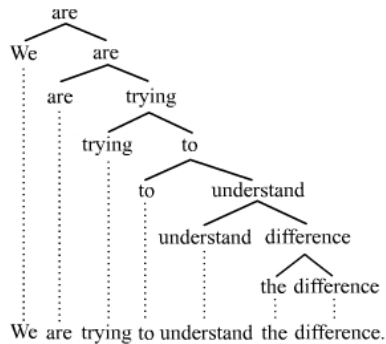
Gramáticas de dependência

- Relações **binárias assimétricas** entre palavras: cabeça $\rightarrow$ dependente
- Não assume constituintes — mais adequada para línguas de ordem livre
- Universal Dependencies (UD): esquema padronizado para 100+ línguas

Gramáticas e estruturas de dependência



Dependency



Constituency (BPS)



Semântica composicional

- Princípio de Frege: o significado de uma expressão é determinado pelo significado das suas partes e pela forma como se combinam
- “todo gato dorme” $\Rightarrow$
 $\forall x. \text{gato}(x) \Rightarrow \text{dorme}(x)$
- Base para sistemas de **pergunta-resposta** e verificação de fatos

Semântica distribucional

- Hipótese distribucional: palavras que aparecem nos mesmos contextos têm significados similares
- “You shall know a word by the company it keeps” — Firth (1957)
- Base teórica de **Word2Vec**, **GloVe** e **embeddings modernos**



Semântica lexical

- Relações entre palavras: sinonímia, antonímia, hiperonímia
- **WordNet** organiza substantivos em hierarquias de *synsets*
- Desambiguação do sentido (WSD): “banco” financeiro vs. assento

Da semântica distribucional aos LLMs

Word2Vec captura relações: $\vec{\text{rei}} - \vec{\text{homem}} + \vec{\text{mulher}} \approx \vec{\text{rainha}}$. GPT e BERT generalizam esse princípio com atenção contextualizada.



Listing 2: gensim: treinando Word2Vec

```
1 from gensim.models import Word2Vec  # pip install gensim
2
3 sentencas = [
4     ["gato", "dorme", "sofá"],
5     ["cachorro", "corre", "parque"],
6     ["gato", "cachorro", "animais"]
7 ]
8
9 modelo = Word2Vec(sentencas, vector_size=50,
10                  window=3, min_count=1, epochs=100)
11
12 # Similaridade semântica
13 print(modelo.wv.similarity("gato", "cachorro"))
14
15 # Palavras mais próximas
16 print(modelo.wv.most_similar("gato", topn=3))
```



Listing 3: gensim: analogia vetorial e PCA

```
1 # Analogia: rei - homem + mulher approx. rainha
2 resultado = modelo.wv.most_similar(
3     positive=["rei", "mulher"], negative=["homem"])
4 print(resultado[:3])
5
6 # Visualizar embeddings em 2D com PCA
7 from sklearn.decomposition import PCA
8 import matplotlib.pyplot as plt
9
10 palavras = list(modelo.wv.key_to_index.keys())
11 vetores = modelo.wv[palavras]
12 coords = PCA(n_components=2).fit_transform(vetores)
13 plt.scatter(coords[:, 0], coords[:, 1], alpha=0.6)
14 for i, p in enumerate(palavras):
15     plt.annotate(p, coords[i], fontsize=8)
16 plt.title("Embeddings Word2Vec - 2D (PCA)")
```

Pesquisas atuais em PLN



Transformers e LLMs

- **Atenção multi-cabeça** substitui recorrência — paralelizável
- BERT (2018): pré-treino bidirecional em texto mascarado
- GPT (2018–): geração autorregressiva em escala
- LLMs com bilhões de parâmetros emergem com capacidades inesperadas (*in-context learning*)

Problemas em aberto

- **Alucinação**: modelos geram fatos falsos com confiança
- **Raciocínio**: LLMs ainda falham em lógica formal e matemática elementar
- **Multilinguismo**: desempenho cai drasticamente em línguas de baixo recurso

PLN em português

- **BERTimbau**: BERT pré-treinado em 2,7 bilhões de palavras em português
- **Maritaca AI**: LLM brasileiro com foco em português e legislação
- **NILC**: grupo de pesquisa em PLN da USP/UFSCar — recursos, corpus e ferramentas abertas
- **Oportunidade**: dados e modelos para português ainda são escassos — área em crescimento

O que é uma série temporal?



Definição

- Sequência de observações **ordenadas no tempo**: $x_1, x_2, \dots, x_T$
- A ordem importa — embaralhar destrói a estrutura
- Exemplos: temperatura diária, preço de ações, demanda de energia, número de usuários ativos

Componentes

- **Tendência (trend)**: direção de longo prazo (crescimento, queda)
- **Sazonalidade**: padrão periódico e previsível (semanal, anual)
- **Ciclo**: flutuação de longo prazo sem período fixo
- **Ruído**: variação aleatória irreduzível

O que é uma série temporal?



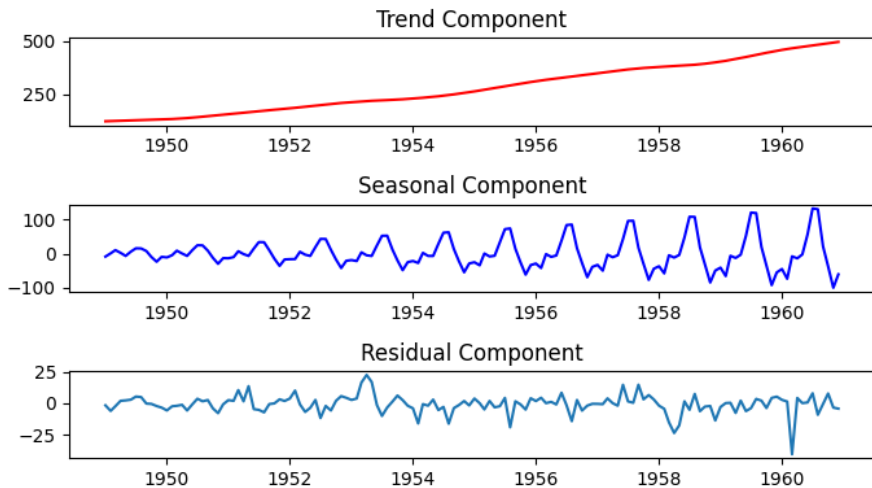
Por que é diferente de dados tabulares?

- Observações **não são independentes** — o passado informa o futuro
- Vazamento temporal: usar dados futuros para prever o passado
- Validação cruzada requer **respeito à ordem cronológica**

Decomposição

Uma série pode ser aditiva: $x_t = T_t + S_t + R_t$, ou multiplicativa: $x_t = T_t \times S_t \times R_t$.

Decomposição de séries temporais



Listing 4: statsmodels: STL e decomposição clássica

```
1 import pandas as pd
2 from statsmodels.tsa.seasonal import STL, seasonal_decompose
3
4 serie = pd.read_csv("passageiros.csv",
5                     index_col="data", parse_dates=True)["passageiros"]
6
7 # Decomposição aditiva clássica
8 decomp = seasonal_decompose(serie, model="additive", period=12)
9 decomp.plot()
10
11 # STL - mais robusta a outliers
12 stl = STL(serie, period=12, robust=True)
13 res = stl.fit()
14 res.plot()
15
16 print(f"Tendência final: {res.trend.iloc[-1]::.0f}")
17 print(f"Amplitude sazonal: {res.seasonal.max()::.0f}")
```



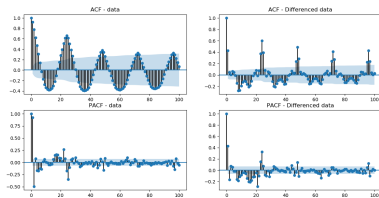
Estacionaridade

- Série **fracamente estacionária**: média, variância e autocovariância constantes ao longo do tempo
- ARIMA e modelos clássicos *exigem* estacionaridade
- Teste ADF (*Augmented Dickey-Fuller*): H_0 = raiz unitária (não estacionária)
- Transformações para estacionarizar: **diferenciação**, log, Box-Cox

Autocorrelação

- **ACF** (função de autocorrelação): correlação de x_t com x_{t-k} para cada lag k
- **PACF** (parcial): correlação direta no lag k , removendo efeitos de lags intermediários
- ACF decai lentamente $\Rightarrow$ não estacionária
- Padrão ACF/PACF guia a escolha de p , d , q no ARIMA

Estacionaridade e autocorrelação





Listing 5: statsmodels: ADF, diferenciação e ACF/PACF

```
1 from statsmodels.tsa.stattools import adfuller
2 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
3
4 # Teste ADF - H0: raiz unitária (não estacionária)
5 resultado = adfuller(serie)
6 print(f"ADF stat: {resultado[0]:.3f} p-valor: {resultado[1]:.4f}")
7 # p < 0.05 → rejeita H0 → série estacionária
8
9 # 1ª diferença para remover tendência
10 serie_diff = serie.diff().dropna()
11 adf2 = adfuller(serie_diff)
12 print(f"Após diff - p-valor: {adf2[1]:.4f}")
13
14 # ACF e PACF da série diferenciada
15 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
16 plot_acf(serie_diff, lags=24, ax=axes[0])
17 plot_pacf(serie_diff, lags=24, ax=axes[1])
```

ARIMA — modelo clássico de previsão



Componentes do ARIMA(p, d, q)

- **AR**(p): autorregressivo — x_t depende de p valores passados

$$x_t = \phi_1 x_{t-1} + \cdots + \phi_p x_{t-p} + \varepsilon_t$$

- **I**(d): integrado — diferencia d vezes para estacionarizar
- **MA**(q): médias móveis — x_t depende de q erros passados

$$x_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

Como escolher p, d, q ?

- d : número de diferenciações até estacionarizar (teste ADF)
- p : lag onde a **PACF** corta abruptamente
- q : lag onde a **ACF** corta abruptamente
- Alternativa automática: `auto_arima` (pmdarima) minimiza AIC/BIC



Listing 6: statsmodels + pmdarima: ajuste e previsão

```
1 from statsmodels.tsa.arima.model import ARIMA
2 import pmdarima as pm    # pip install pmdarima
3
4 # Seleção automática de ordem
5 modelo_auto = pm.auto_arima(serie_treino,
6                             seasonal=True, m=12,
7                             information_criterion="aic",
8                             stepwise=True)
9 print(modelo_auto.summary())
```



Listing 7: statsmodels + pmdarima: ajuste e previsão

```
1  # Ajuste manual com ordem escolhida
2  modelo = ARIMA(serie_treino, order=(1, 1, 1),
3                seasonal_order=(1, 1, 0, 12))
4  res     = modelo.fit()
5
6  # Previsão com intervalo de confiança
7  previsao = res.get_forecast(steps=12)
8  media    = previsao.predicted_mean
9  ic       = previsao.conf_int(alpha=0.05)
10
11 # Métrica
12 from sklearn.metrics import mean_absolute_percentage_error
13 mape = mean_absolute_percentage_error(serie_teste, media)
14 print(f"MAPE: {mape:.2%}")
```




Facebook Prophet

- Modelo aditivo:
$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t$$
- $g(t)$: tendência (linear ou logística)
- $s(t)$: sazonalidade via séries de Fourier
- $h(t)$: feriados e eventos especiais
- Robusto a dados faltantes e mudanças de tendência (*changepoints*)
- Interface simples — ideal para analistas sem background em séries temporais

Outros modelos relevantes

- Exponential Smoothing (ETS): médias ponderadas exponencialmente decrescentes
- Gradient Boosting (XGBoost/LightGBM): features de lag como entrada tabular — muito competitivo
- N-BEATS / N-HiTS: redes neurais puramente para séries, sem recorrência
- TimesFM / Chronos: LLMs pré-treinados em séries temporais (zero-shot)

Listing 8: Facebook Prophet: ajuste e previsão

```
1 from prophet import Prophet  # pip install prophet
2 import pandas as pd
3
4 # Prophet exige colunas "ds" (datetime) e "y" (valor)
5 df_prophet = serie.reset_index()
6 df_prophet.columns = ["ds", "y"]
7
8 modelo = Prophet(yearly_seasonality=True,
9                  weekly_seasonality=False,
10                 changepoint_prior_scale=0.1)
11 modelo.add_country_holidays(country_name="BR")
12 modelo.fit(df_prophet[df_prophet["ds"] < "2024-01-01"])
13
14 futuro    = modelo.make_future_dataframe(periods=12, freq="MS")
15 previsao  = modelo.predict(futuro)
16 modelo.plot(previsao)
17 modelo.plot_components(previsao)
```



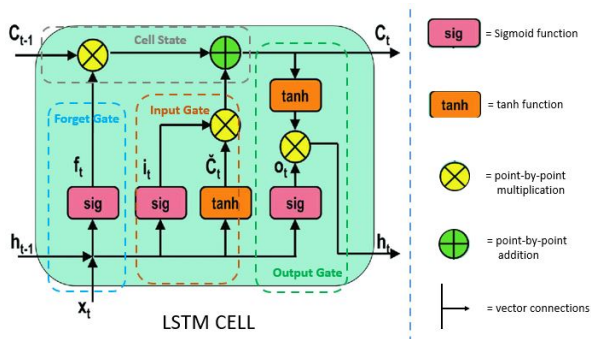
Por que LSTM?

- Redes recorrentes (RNN) sofrem de **gradiente que desaparece** — esquecem dependências longas
- LSTM (*Long Short-Term Memory*) resolve com **portas** que controlam o fluxo de informação
- Porta de esquecimento, de entrada e de saída — a rede aprende *o que* lembrar
- Captura padrões não-lineares e dependências de longo prazo

Preparação dos dados

- Criar janelas deslizantes:
 $[x_{t-w}, \dots, x_{t-1}] \rightarrow x_t$
- Normalizar para $[0, 1]$ — LSTM é sensível à escala
- Respeitar a ordem cronológica no split treino/teste

LSTM para séries temporais





Listing 9: Keras: janelas deslizantes

```
1 import numpy as np
2 from sklearn.preprocessing import MinMaxScaler
3
4 scaler = MinMaxScaler()
5 valores = scaler.fit_transform(serie.values.reshape(-1, 1))
6
7 def criar_janelas(dados, janela=12):
8     X, y = [], []
9     for i in range(len(dados) - janela):
10         X.append(dados[i:i+janela, 0])
11         y.append(dados[i+janela, 0])
12     return np.array(X), np.array(y)
13
14 X, y = criar_janelas(valores, janela=12)
15 split = int(len(X) * 0.8)
16 X_tr, X_te = X[:split], X[split:]
17 y_tr, y_te = y[:split], y[split:]
```



Listing 10: Keras: reshape e definição do modelo

```
1  # Reshape: (samples, timesteps, features)
2  X_tr = X_tr.reshape(-1, 12, 1)
3  X_te = X_te.reshape(-1, 12, 1)
4
5  from tensorflow.keras.models import Sequential
6  from tensorflow.keras.layers import LSTM, Dense, Dropout
7
8  modelo_lstm = Sequential([
9      LSTM(64, return_sequences=True, input_shape=(12, 1)),
10     Dropout(0.2),
11     LSTM(32),
12     Dense(1)
13 ])
14 modelo_lstm.compile(optimizer="adam", loss="mse")
15 modelo_lstm.fit(X_tr, y_tr, epochs=50,
16                 batch_size=16, validation_split=0.1)
```



Listing 11: Keras: treinamento e inversão da escala

```
1 modelo_lstm.compile(optimizer="adam", loss="mse")
2 modelo_lstm.fit(X_tr, y_tr,
3                 epochs=50, batch_size=16,
4                 validation_split=0.1, verbose=0)
5
6 # Previsão e inversão da normalização
7 y_pred      = modelo_lstm.predict(X_te)
8 y_pred_inv  = scaler.inverse_transform(y_pred)
9 y_te_inv    = scaler.inverse_transform(y_te.reshape(-1, 1))
10
11 from sklearn.metrics import mean_absolute_percentage_error
12 mape = mean_absolute_percentage_error(y_te_inv, y_pred_inv)
13 print(f"MAPE LSTM: {mape:.2%}")
```

Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!